

RepFinder

Plataforma para centralizar a busca por vagas em repúblicas universitárias, conectando estudantes que procuram moradia a representantes que divulgam oportunidades.

O problema

A busca por moradia universitária ainda acontece de forma majoritariamente informal — grupos de WhatsApp, indicações de colegas, publicações espalhadas em redes sociais. Para quem procura, faltam informações organizadas. Para quem oferece, faltam ferramentas para gerenciar candidatos de forma estruturada.

Esse cenário gera alguns problemas recorrentes:

- Vagas divulgadas em canais fragmentados e de difícil acesso
- Nenhum histórico ou rastreabilidade nas candidaturas
- Processo de seleção informal, sem critérios claros
- Comunicação descentralizada entre estudantes e representantes

A proposta

O RepFinder organiza esse processo em uma plataforma estruturada com dois perfis de usuário:

- **Estudantes** — buscam vagas disponíveis e se candidatam de forma padronizada
- **Representantes** — divulgam vagas, acompanham candidaturas e gerenciam o processo de seleção

Estrutura do projeto

```
repfinder/  
├── api/           # Backend REST (Node.js + Hono + TypeScript)  
│   └── docs/      # Documentação técnica, schema do banco e coleção Insomnia  
├── apps/         # Aplicativos Flutter (em desenvolvimento)  
└── docs/         # Enunciado do projeto
```

Como rodar a API

Pré-requisitos: Node.js 18+, MariaDB/MySQL rodando localmente.

```
cd api
cp .env.example .env # preencha as variáveis abaixo
npm install
npm run dev
```

Variáveis de ambiente necessárias no `.env`:

```
DB_HOST=SEU_HOST
DB_USER=SEU_USER
DB_PASS=SUA_SENHA
DB_NAME=SEU_BANCO
JWT_SECRET=SEU_SECRET
```

O banco de dados e as tabelas são criados automaticamente na primeira execução.

A API sobe em `http://localhost:3030`. A coleção de testes está em `api/docs/Insomnia_RepFinder_API.json`.

Documentação técnica

A documentação da API — contexto do problema, decisões arquiteturais, regras de negócio e schema do banco — está em `api/docs/`.

Arquitetura da Aplicação

A arquitetura do RepFinder foi pensada para refletir diretamente o fluxo de interação entre os diferentes tipos de usuários e o sistema, mantendo uma separação clara entre responsabilidades e preparando o terreno para evoluções futuras.

Clientes

A aplicação é consumida por dois clientes distintos, ambos desenvolvidos em Flutter:

- um aplicativo voltado para **estudantes**, responsável por buscar vagas e enviar candidaturas
- um aplicativo voltado para **representantes**, responsável por gerenciar vagas e avaliar candidatos

Apesar de atenderem perfis diferentes, ambos interagem com o mesmo backend por meio de requisições HTTP, utilizando autenticação baseada em token JWT.

Backend

O backend atua como ponto central de orquestração, sendo responsável por:

- autenticação dos usuários
- aplicação das regras de negócio
- controle de acesso aos recursos
- persistência e recuperação de dados

A comunicação com os clientes ocorre via API REST, onde cada requisição passa por um fluxo bem definido:

1. **Recebimento da requisição HTTP**
 2. **Validação de autenticação (JWT via Bearer Token)**
 3. **Encaminhamento para o módulo responsável**
 4. **Execução das regras de negócio**
 5. **Acesso à camada de dados**
 6. **Retorno da resposta ao cliente**
-

Organização por Módulos

O sistema é dividido em três módulos que representam os principais domínios da aplicação:

- **users** — autenticação, cadastro e gerenciamento de perfil
- **vacancies** — criação e gestão de vagas por representantes
- **applications** — envio e acompanhamento de candidaturas por estudantes

Cada módulo encapsula sua própria lógica, reduzindo acoplamento e facilitando manutenção independente.

Autenticação

O acesso às rotas protegidas é controlado por um middleware de autenticação que valida tokens JWT enviados pelos clientes no header `Authorization: Bearer <token>`.

Esse mecanismo garante que:

- apenas usuários autenticados acessem recursos protegidos
- a identidade do usuário esteja disponível durante todo o ciclo de processamento da requisição

Senhas são armazenadas com hash PBKDF2-SHA512 com salt aleatório e comparação via `timingSafeEqual`, eliminando vulnerabilidades de timing attack.

Camadas Internas

Dentro de cada módulo, a aplicação segue uma divisão em camadas que organiza o fluxo de execução:

- **Routes** — interface HTTP: recebe a requisição, delega ao service, retorna a resposta
- **Service Layer** — concentra as regras de negócio e decisões de domínio
- **Repository Layer** — comunicação direta com o banco de dados via SQL

Essa separação evita mistura de responsabilidades e mantém o código previsível e testável.

Persistência

A persistência é realizada em banco de dados relacional MariaDB, com as seguintes entidades:

- **users** — dados de cadastro e autenticação
- **vacancies** — vagas criadas por representantes
- **applications** — candidaturas vinculando estudantes a vagas

A comunicação com o banco utiliza pool de conexões via `mysql2`, com queries parametrizadas para prevenção de SQL Injection.

Comunicação Assíncrona (Sprint 2)

A arquitetura prevê a introdução de um componente de mensageria baseado em **Redis Pub/Sub**, desacoplando o processamento de eventos da API principal.

O fluxo previsto é:

1. A API publica eventos no Redis após operações relevantes
2. Um **Worker** independente consome esses eventos
3. O Worker notifica os clientes via **SSE (Server-Sent Events)**

Eventos previstos:

- `application.created` — publicado quando um estudante se candidata a uma vaga
- `application.status.updated` — publicado quando o representante aceita ou rejeita uma candidatura

O uso de SSE é preferível a WebSockets neste contexto pois o canal de notificação é unidirecional — o servidor notifica o cliente, sem necessidade de comunicação no sentido inverso.

Diagrama Arquitetural

O diagrama abaixo representa os componentes da arquitetura e seus protocolos de comunicação:

- **Clientes** → **API**: HTTP/REST com autenticação JWT Bearer
- **API** → **MariaDB**: TCP via pool MySQL (mysql2)
- **API** → **Redis**: Redis Pub/Sub (publish)
- **Redis** → **Worker**: Redis Pub/Sub (subscribe)
- **Worker** → **Clientes**: SSE (Server-Sent Events)